

S5 explained — documents

A full walkthrough of what was built, why, and what to say when asked.

Throwaway file. Move it into **docs/** if you want to keep it, or delete it.

1. What S5 is

The brief asks for one thing, in two sentences that are easy to read as one:

Upload Word, PDF, and PowerPoint files as source materials in the room **as context**. The AI tutor **can query those as needed**.

The student **must be able to use** these common school files as source material. Build a real ingestion path that extracts accurate content. Handle bad and unsupported inputs deliberately.

Read carefully, that is three separate requirements:

1. **Ingest** three formats, accurately.
2. Let the tutor **query** them — *as needed*, not always.
3. Handle bad files **deliberately** — a typed reason, never a crash.

The third is scored as heavily as the first. "Deliberately" is the word that turns a **try/except: pass** into a design.

The one-sentence version of the feature: a student uploads a handout, and the tutor can quote it back with the page number.

2. The shape of the solution

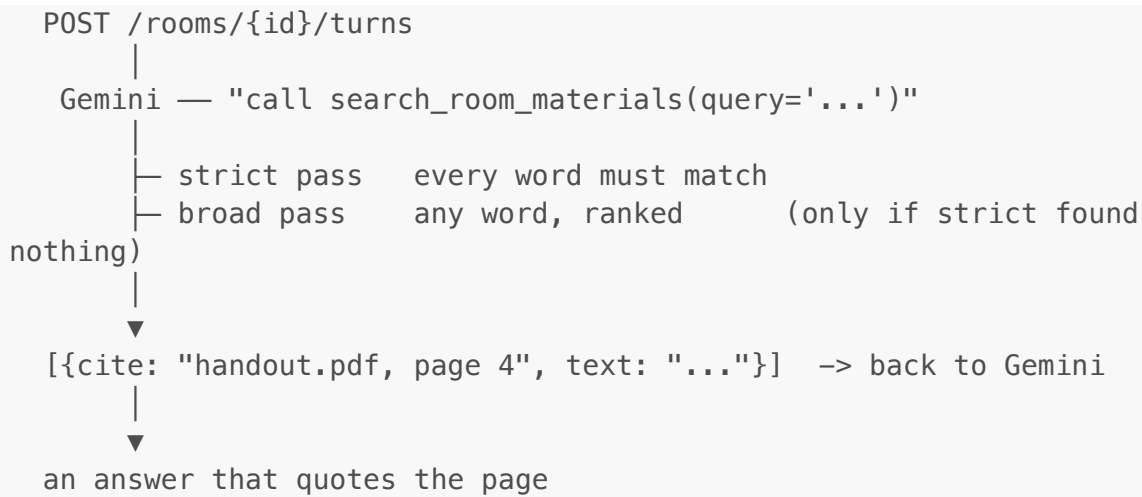
UPLOAD (one request, once per file)

POST /rooms/{id}/documents

— detect	extension + first 4 bytes + size	-> 415 / 413
— sha256	seen before? return it, do nothing	
— extract	pdf / docx / pptx -> [Page]	-> 422 x 5
— chunk	250 words, 40 overlap, never cross a page	
— insert	Postgres computes the tsvector on write	
— evict	keep 10 files per room, oldest out	

▼
201, and the tutor can already search it

ASK (any later request, any number of times)



The key idea: the file is read once, by us, and never sent to the model. The model retrieves five passages when it needs them.

3. Why the tutor pulls instead of us pushing

The obvious alternative is to paste the document into the prompt. Measured, on real files:

	Tokens
A 4.4 MB textbook, whole	1,454,615
One search result (5 passages)	1,322

gemini-2.5-flash has a 1 M token context window. **That book does not fit.** A full 20 MB of text is roughly 6.6 M tokens, six times over.

So retrieval is not an optimisation here. It is the only thing that works at all. And the cost is flat: five passages whether the room holds one page or a 300-page textbook.

The brief agrees, in its own words: *"can query those **as needed**".*

4. Detection — what is this file, really?

app/documents/detect.py

A filename is a string the client chose. Renaming **virus.exe** to **notes.pdf** changes nothing about the contents, and handing those bytes to a parser to find out is the wrong order. So we check both:

```
_ZIP = b"PK\x03\x04"
_MAGIC = {PDF: b"%PDF", DOCX: _ZIP, PPTX: _ZIP}
```

- the **extension** says what the file claims to be
- the **first four bytes** say what it is

Disagreement is a **415**. That catches the honest mistake and the dishonest one through the same code path.

The limit, stated rather than glossed over

.docx and **.pptx** are **both ZIP archives**, so both start with **PK**. The magic bytes can prove a file *is* a ZIP and cannot prove which Office format it holds. A PowerPoint renamed **.docx** gets past detection and fails one step later, in the extractor, as **CORRUPT_ARCHIVE**.

Say this out loud on the call. "We check magic bytes" sounds stronger than it is; knowing exactly where the check stops is the part that reads as engineering.

Why not **python-magic**

It is a C library and another system package in the image, to identify three formats whose signatures are four bytes long and have not changed in twenty years. The dependency costs more than the code it replaces.

5. Extraction — the text, and which page it came from

app/documents/extract.py

Every extractor returns the same thing, a list of **Page**, so everything downstream is written once:

```
@dataclass(frozen=True)
class Page:
    number: int | None    # 1-based, or None where the format has no such
    thing
    text: str
```

Why Word has no page number

A PDF stores pages. A PowerPoint stores slides. **A .docx stores neither** — it holds a stream of paragraphs, and page breaks are computed by whatever renders it, from the font metrics and paper size in use at the time. Open the same file on another machine and "page 4" can be a different paragraph.

So **page_count** is **NULL** for Word and a citation is the filename alone.

This is the answer to "why is this field null?" and it is worth giving properly: inventing a number would be worse than omitting one. The tutor would quote it, the student would turn to page 4, and it would not be there. A missing citation is a gap; a confident wrong one is a lie.

What each format gives

- **PDF** — text per page, via PyMuPDF.
- **Word** — paragraphs and tables, walked in document order. Not `document.paragraphs` then `document.tables`, which would move every table to the end and separate it from the text explaining it. `_iter_docx_blocks` walks the XML body instead.
- **PowerPoint** — every shape's text, tables, and **speaker notes**. Notes are often the best study content on a slide: the slide says "Photosynthesis", the notes say what it is.

Tables become one line per row, joined with `|`, so a hit inside a table carries its row. A lone cell reading `42` tells the student nothing; `Carbon | 12 | 6` does.

What is not extracted, and why the list is written down

Every gap is **silent** — no error, no warning, just a document that quietly does not contain what the student can see on screen. The first sign of trouble is the tutor saying the material does not cover something it visibly does.

Format	Lost
<code>.docx</code>	headers, footers, text boxes, footnotes, endnotes, comments, chart data
<code>.pptx</code>	SmartArt, chart data, WordArt, images
<code>.pdf</code>	images, image-rendered formulas; two-column layouts can interleave

The Word row is the one that bit us in testing — see §12.

6. Chunking

`app/documents/chunking.py`

A whole document is the wrong unit to search. Ask "what did the handout say about photosynthesis" and a match on a 40-page PDF tells you the answer is *somewhere* in it, which is not an answer.

Three rules:

~250 words per chunk. Small enough that a hit points at a paragraph rather than a chapter; large enough that a sentence keeps the context that gives it meaning.

40 words of overlap. A boundary landing mid-explanation would leave neither side making sense. Overlapping means any short passage appears whole in at least one chunk. Costs ~15% more rows.

Never across a page boundary. This is the one to lead with. A chunk carries one page number, and that is what the tutor cites. A chunk spanning pages 3 and 4

could only claim one of them, so half its text would be cited to the wrong place. Pages are chunked independently even when that leaves a short one.

Why words, not tokens

A token count needs the model's tokeniser — a second dependency answering a question we do not have. Nothing here is sized against a hard model limit; the search tool returns a fixed number of chunks and *that* is where the budget is enforced. Words are close enough and anyone can count them.

The subtle bit

The last window can be entirely overlap: when the word count divides evenly by the step, the final slice repeats the tail of the previous chunk and adds nothing. It is dropped, or it would match twice for no reason.

7. Indexing

`app/models/document.py`

Two tables. `documents` answers "what did I upload"; `document_chunks` answers "where does this sentence appear".

```
tsv: Mapped[str] = mapped_column(
    TSVECTOR,
    Computed("to_tsvector('english', text)", persisted=True),
    nullable=False,
)
```

A generated column, maintained by Postgres. It cannot drift from the text it summarises, because there is no code path that updates one without the other — there is no code path that updates it at all.

Worth checking rather than assuming when the migration was written: had Alembic emitted a plain column, every row would have inserted with a NULL tsvector and search would have matched nothing *while looking perfectly healthy*.

The GIN index on `tsv` is what makes search a lookup rather than a scan: it indexes each lexeme separately.

What is deliberately absent

The file itself. We store the SHA-256, name and size — not the bytes. Nothing reads them back, so keeping them means a volume or object store, a lifecycle to clean it up, and a second copy of something already processed. If "download the original" becomes a feature it brings its own storage with it.

A processing status. Upload is synchronous, so a row exists only if the file was read. Its existence *is* the status. A column nothing ever sets to **failed** is dead weight.

Why SHA-256 of the whole file is fine

Measured: **8.9 ms for 20 MB**, at 2.3 GB/s. Against ~14 s to parse a 20 MB PDF that is 0.06% of the work, and the bytes are already in memory. Hashing a prefix plus the size would save 9 ms and collide between files from the same template.

Re-uploading identical bytes returns the existing document. Students re-upload — they lose the tab, they are not sure it worked. A **409** for that would be technically defensible and useless.

8. Search — strict, then broad

`app/services/document.py`

```
hits = await _run(db, room, func.websearch_to_tsquery(CONFIG, text), limit)
if hits:
    return hits
return await _run(db, room, _any_term_query(text), limit)
```

`websearch_to_tsquery` rather than `to_tsquery`, because the query string is written by a language model: `to_tsquery` demands operator syntax and raises on anything else, so `photosynthesis and light` would be a 500. The `websearch` form takes plain language, understands quoted phrases and `-excluded`, and never raises.

But it joins terms with AND, and that is the bug that made the whole feature look broken in a real room:

```
'protections prevent arbitrary tool calls'
-> 'protect' & 'prevent' & 'arbitrari' & 'tool' & 'call'
```

The document said "protections" and "arbitrary" and never said "prevent". **One ordinary English word reduced five good matches to zero.** The tutor read the silence as "your materials do not cover this" and answered from general knowledge — fluently, plausibly, and without the handout it was holding. Nothing errored. The only symptom was a vague answer.

So: strict first (precision, and it honours quoted phrases), broad on empty (recall, ranked by `ts_rank`).

Why there is no relevance threshold

Because measured on real data the ranks overlap:

	best rank	avg rank
a good query	0.0649	0.0320
a nonsense query	0.0274	0.0174

A good query's *average* sits below a nonsense query's *best*. Any fixed cut-off would be a magic number tuned to one document.

So relevance is judged where meaning is understood rather than where words are counted: the prompt tells the model to read each passage, use it only if it answers the question, and say the material does not cover it otherwise. **A forced citation the student can look up and not find is worse than no citation.**

The boundary of the whole technique

Asked for a report's submission date, the tutor honestly said it could not find one. The date was on slide 1, reading **AUGUST 2026**. The query terms were *report, summary, submission, date*.

A date does not contain the word "date".

No ranking change reaches that. Lexical search matches words and has no concept that **AUGUST 2026** is a date, that "how plants eat" is photosynthesis, or that "the method they used" is a methodology section. That is the case embeddings solve, and it is why hybrid retrieval is high on the more-time list.

This is a strong thing to volunteer on the call: a query class the technique provably cannot serve, with evidence.

9. The tool

`app/tools/search_materials.py`

Third tool, same shape as S4's two. Two things worth pointing at.

The room is not an argument. `context.room` was resolved from the URL before the model saw anything. There is no `room_id` parameter for it to fill in, so there is no way to read another room's material — the same property as `query_study_history` taking no student.

Results carry a finished citation, not an id.

```
return {"passages": [{"cite": hit.citation, "text": hit.text} for hit in hits]}
```

"algebra-handout.pdf, page 4" is handed to the model as a string to repeat. Giving it an opaque id and asking it to render a reference is a step it can get wrong; giving it the finished string is a step it cannot. You can see this in the inspection endpoint — the citation in the answer is visibly the one it was given.

10. Uploading — and why it is synchronous

`app/api/routes/documents.py`, `app/services/document.py`

Three endpoints: upload, list, delete.

The interesting decision is that reading, chunking and indexing all happen **inside the request**. The obvious alternative is `202 Accepted` and a background task.

The reason is the brief's own requirement to handle bad input deliberately. Extraction is *where most bad input is discovered* — encrypted, corrupt, no text layer, undecodable, empty. Four of the six cases cannot be known until a parser tries. In the background, every one becomes a status field the client polls and then has to interpret. In the request, each is an HTTP status with a code.

Three things fall out for free:

- **201 means searchable now**. No race where a student uploads then immediately asks a question before indexing finished.
- **No background session**. A task outliving its request needs its own database session — a classic source of "works locally, leaks in production".
- **No flaky tests**. Nothing to poll or wait on.

The cost, measured

Text-heavy PDF parses at about **1.5 MB/s**. So 4.4 MB takes ~3 s and a 20 MB worst case around **14 s**. That is a long HTTP request. It survives typical 30–60 s proxy timeouts without much room, and a dropped connection at 13 s loses the whole upload.

Parsing runs in `asyncio.to_thread`, because it is the one genuinely CPU-bound thing the app does and inline it would hold the event loop for that whole time.

When the decision flips

Not file size — **OCR**. OCR is seconds per page, times 148 pages, and no HTTP request survives that. At that point the queue, the status column and the polling endpoint all arrive together. So OCR is a change to the *shape* of the feature, not an addition to it. That is the honest form of the answer: right for these constraints, and here is exactly what breaks it.

11. The per-room limit

A room keeps **10 files**. Going past that does not refuse the upload — it removes the oldest and names it:

```
{ "filename": "newest.pdf", "chunk_count": 4, "evicted": ["note0.pdf"] }
```

Eviction over blocking because a student part-way through studying should not hit a wall, and the file they are reaching for now matters more than one from last term. The number is bounded by what the prompt can carry — every filename is listed there (§12), and fifty would cost more context than the answers it is meant to improve.

evicted is returned rather than only logged, because deleting a student's file is not a detail. They should be told by the request that did it, not discover it weeks later when the tutor stops citing something.

Oldest-by-upload, not least-recently-used. LRU would be the better rule — a reference uploaded in March and read weekly beats one added in June and never opened — but nothing records when a document was last searched, and a write on every search to find out is a worse trade. Upload order is at least predictable, which is what makes it safe to describe to a student.

12. Telling the tutor the room has files

This is the fix for a failure that looked like a broken tool and was not.

A student uploaded a 20-slide thesis deck and asked *"give me a summary about the thesis"*. The answer was a general essay about what a thesis is, with an example about school lunches. **tool_calls**: [].

Reading the stored **rendered_prompt** explains it completely. The prompt took four variables — education level, room title, recent turns, the message — and **mentioned documents nowhere**. Not their names, not their count, not their existence. The only reference was the abstract tool description.

So the model had to *guess* whether this room contained anything worth searching. It guessed no. It had no way to guess yes.

v6 adds:

```
## What is in this room
```

```
The student has uploaded these files:
```

```
- 2003013.pptx (20 slides)
```

```
These exist. Use `search_room_materials` to read them whenever the question could
```

touch what is in them – and treat a vague reference as pointing here. "The thesis", "the assignment", "our notes" mean **these files**, not the idea in general.

Names and sizes only. The deck measured 5,843 tokens; carrying contents on every turn would nearly quadruple the prompt to answer "thanks", and a room with five handouts would not fit. Thirty tokens buy the thing that was actually missing — the model knowing there is something to look at.

After the change, the same room answered:

"Here are the four budgets and their default values (S4-explanation.pdf, page 4): *max_agent_iterations* 5, *max_tool_calls_per_turn* 6, *tool_timeout_seconds* 8, *turn_deadline_seconds* 45."

13. Bad files — all seven

Each gets the right status and a stable *code*. Never a 500.

Status	<i>code</i>	Cause
415	UNSUPPORTED_TYPE	Not a supported type, or contents disagree with the name
413	FILE_TOO_LARGE	Over 20 MB
422	CORRUPT_ARCHIVE	Damaged, or not really the format it claims
422	ENCRYPTED	Password-protected
422	NO_TEXT_LAYER	A scan — images of text
422	UNDECODABLE_TEXT	Fonts record shapes, not characters
422	EMPTY_EXTRACTION	Opens fine, contains no words

Why the last three are not one error

All three yield no usable text and it would be easy to collapse them. They are kept apart because **the student's next action differs**: a scan needs OCR we do not do, an undecodable file needs re-exporting from the original, and an empty one never had anything in it.

Why a scan is rejected rather than accepted as empty

An accepted scan sits in the room looking searchable, matches nothing, and gives no clue why. Rejecting it clearly *is* the deliberate handling the brief asks for. Silently saving an empty document is the real failure.

The fixtures are built, not committed

`tests/documents.py` constructs every file in memory with the same libraries that read them back. A committed `scanned.pdf` is opaque — you cannot tell what makes it a scan without opening it. Built, the good and bad cases are visibly the same construction with one thing changed.

14. Real bugs found while building

The most useful section. Every one was found by **running** the system, not reading it.

A 500 from a real 700 KB PDF

```
asyncpg.exceptions.CharacterNotInRepertoireError:
  invalid byte sequence for encoding "UTF8": 0x00
[parameters: (... , '\x00\x02\x01\x04\x03 \x05\x07\x06 ...')]
```

Some PDFs embed fonts with no `ToUnicode` map, so there is no way back from a glyph to a character and the extractor returns raw font indices. Postgres cannot store `\x00` in a `text` column at all.

Two fixes, because they are two problems. Everything extracted is now stripped of characters that cannot survive the database — at the one point every extractor's output passes through, so no *future* unknown case can 500 the same way. And a document that is *mostly* such characters is refused rather than indexed as noise.

The stripping has a trap. The obvious implementation is "drop every character in Unicode category `C`", and it is wrong here: category `Cf` holds the zero-width joiner, which Bangla, Hindi and Arabic need. `ज़` and `र्ज` are different words. Only `Cc` and `Cs` are dropped, and a test asserts the joiner survives.

Then the follow-up question — was rejecting that file *correct*? Checked rather than assumed: **98 Type 3 fonts**, 138 of 148 pages over the threshold, the other 10 blank, and `text/blocks/words` extraction modes all returning the same indices. Two independent libraries, PyMuPDF and pypdf, produced byte-identical garbage. The file contains drawings of letters, not letters. Rejecting it is right; reading it needs OCR.

The prompt that was written and never switched on

`app/services/turn.py` pins the prompt version as a constant. `v4.md` was written — the one that introduces `search_room_materials` — and the constant still said **3. v3's tools section says "You have two tools."**

So the declaration was sent to Gemini and the prompt talked it out of existence. Every turn ran on v3 for six hours. Nothing failed; answers were merely worse, and the only evidence was `prompt_version: 3` on a stored attempt.

The version stays pinned, because a prompt change is a behaviour change and should be a reviewed commit. The guard is a test asserting the pinned version is the newest on disk, which fails with:

```
tutor_system v6 exists but the code still uses v5. Bump the constant in
app/services/turn.py, or delete the unused prompt file.
```

Be honest about the weakness: that is the patch, not the cure. The coupling should not exist. Putting `status: active` in the prompt's own front-matter makes adding and switching one edit in one directory.

Search that returned nothing for a good question

Covered in §8. One unmatched word emptied the result.

PowerPoint text that vanished

A group is a shape containing shapes and has no text of its own, so iterating `slide.shapes` and asking each for its text **silently loses everything inside one**. Decks group constantly — labelled diagrams, callouts, figure-plus-caption.

```
before:  'PLAIN TEXTBOX'
after:   'PLAIN TEXTBOX\n\nINSIDE  GROUP  ONE\n\nNESTED TWO DEEP'
```

Found by writing a test with a group in it, not by reading the library docs.

Logging that went nowhere

```
root level: WARNING
app logger effective: WARNING
INFO emitted? False
```

Nothing called `basicConfig.log_level` was a setting **nothing read**, so every `logger.info` in the project — documents, turns, repairs, tool caching — was discarded. The observability existed in the source and nowhere else.

`force=True` matters: uvicorn installs handlers first, and without it the call is a silent no-op.

Eviction ordered by a timestamp that was always the same

Two eviction tests failed, and the cause was not the tests:

```
f0.pdf created_at=2026-08-13 21:50:18.481293+00
f1.pdf created_at=2026-08-13 21:50:18.481293+00  <- identical
```

Postgres `now()` is **transaction start**. Files uploaded in one transaction get the same timestamp and eviction picks its victim by whichever random UUID sorts first. Production happened to be safe — one upload per request — but eviction *deletes data*, and its order must not depend on where transaction boundaries fall. `created_at` is now set in Python at insert.

A fixture that silently truncated itself

The PDF builder used `insert_text`, which draws one unwrapped line and clips anything past the page edge. A fixture quietly missing half its content would make a broken extractor look fine. Now `insert_textbox`, with an assertion that the text fit.

15. Files

New:

<code>app/documents/detect.py</code>	extension + magic bytes + size	(113)
<code>app/documents/extract.py</code>	three extractors, cleaning, ratios	(352)
<code>app/documents/chunking.py</code>	overlapping, page-bounded chunks	(87)
<code>app/services/document.py</code>	upload, evict, describe, search	(380)
<code>app/models/document.py</code>	documents + document_chunks	(131)
<code>app/schemas/document.py</code>	read + upload response	(46)
<code>app/api/routes/documents.py</code>	three endpoints	(116)
<code>app/tools/search_materials.py</code>	the third tool	(75)
<code>prompts/tutor_system/v4.md</code>	adds the tool	
<code>prompts/tutor_system/v5.md</code>	judge relevance; search before recapping	
<code>prompts/tutor_system/v6.md</code>	lists the room's files	
<code>scripts/check_document.py</code>	will this file upload? ask before a demo	
<code>migrations/versions/20260813_1054_documents_and_chunks.py</code>		
<code>tests/documents.py</code>	fixtures built in memory	(138)
<code>tests/test_documents.py</code>	41 tests	(636)

Changed:

<code>app/api/errors.py</code>	<code>UnsupportedFileError</code> , <code>FileTooLargeError</code> , <code>UnreadableFileError</code>
<code>app/services/turn.py</code>	renders the materials list; prompt pin
<code>app/tools/registry.py</code>	registers the third tool
<code>app/main.py</code>	<code>logging.basicConfig</code> from <code>LOG_LEVEL</code>
<code>tests/test_prompts.py</code>	the pinned-version guard
<code>tests/test_tools.py</code>	three tools, not two

232 tests pass. Ruff clean. Migration round-trips with no drift.

16. Questions you should expect

"Walk me through what happens when I upload a file."

Detect, hash, extract, chunk, insert, evict — and say that a `201` means the tutor can already search it, because processing is synchronous. Then say why: extraction is where bad files are discovered, and that makes each one an HTTP status instead of a field to poll.

"Why not process in the background?"

Give the trade honestly, with the number: ~1.5 MB/s, so 14 s worst case at the 20 MB cap, parsed off the event loop. Then say what flips it — OCR, not size — and that the queue, status column and polling endpoint all arrive together when it does.

"How do you know a `.pdf` is really a PDF?"

Extension plus first four bytes. Then immediately give the limit: `.docx` and `.pptx` are both ZIPs, so the signature proves ZIP and not which Office format. A PowerPoint named `.docx` is caught by the extractor as `CORRUPT_ARCHIVE`, and there is a test for exactly that.

"What happens with a scanned PDF?"

Rejected as `NO_TEXT_LAYER`, not accepted as empty — because an accepted scan sits in the room looking searchable and matching nothing. Then mention `UNDECODABLE_TEXT` and the Type 3 font case, which is the more interesting one and shows the list came from real files.

"Why is `page_count` null for Word files?"

Because `.docx` does not contain page numbers; the renderer computes them. Inventing one would send the student to a page that is not there.

"Does the model read the whole document?"

No — zero tokens. Our code reads it. The model sees five passages, ~1,300 tokens, flat regardless of file size. Then the killer number: that 4.4 MB textbook is 1.45 M tokens against a 1 M context window, so stuffing does not merely cost more, it does not work.

"How do you stop it searching another student's files?"

The room is a join, not an argument. There is no `room_id` parameter for the model to set. Same property as the history tool.

"What's the weakest part of this?"

Answer with the date. Lexical search cannot find `AUGUST 2026` from the word "date", and no ranking change fixes it — it is the boundary of word matching. Then the silent extraction gaps: Word headers and footers are invisible to `python-docx`, which is a plausible reason that same report had no findable date. Both are documented with the fix.

"How did you find these problems?"

This is the best question to get. By running it against real files — a 700 KB LaTeX book that returned a 500, a thesis deck that got a school-lunch analogy, a question whose search returned nothing because of one extra word. Every fix in §14 came from a real failure, and each one closed a *class* of failure rather than the instance: a sanitiser at the single choke point, a test that catches any unadopted prompt, a search that degrades to recall instead of silence.